# String

El texto es un tipo de datos de cadena. Cualquier tipo de datos escrito como texto es una cadena. Cualquier dato bajo una **comilla única, doble o triple son cadenas**.

Existen diferentes **métodos de cadena y funciones integradas** para tratar con los tipos de datos de cadena.

- **capitalize**(): Convierte el primer carácter de la cadena en mayúscula.
- **count**(): devuelve las apariciones de subcadena en string, count(substring, start=.., end=..). El inicio es una indexación inicial para contar y el final es el último índice a contar.
- **endswith**(): Comprueba si una cadena termina con un final especificado
- **expandtabs**(): Reemplaza el carácter de la pestaña con los espacios, el tamaño predeterminado de la pestaña es 8. Se necesita argumento de tamaño de pestaña
- **find**(): Devuelve el índice de la primera aparición de una subcadena, si no se encuentran retornos -1
- **rfind**(): Devuelve el índice de la última ocurrencia de una subcadena, si no se encuentran retornos -1
- **format**(): cadena de formatos en una salida más bonita. Más información sobre el formato de cadena comprobar este enlace.
- **index**(): Devuelve el índice más bajo de una subcadena, los argumentos adicionales indican el índice de inicio y finalización (predeterminado 0 y longitud de cadena - 1). Si no se encuentra la subcadena, aumenta un valorError.
- **ridex**(): Devuelve el índice más alto de una subcadena, los argumentos adicionales indican el índice de inicio y finalización (predeterminado 0 y longitud de cadena - 1)
- **isalnum**(): Comprueba el carácter alfanumérico.
- isalpha(): Comprueba si todos los elementos de cadena son caracteres alfabéticos (a-z y A-Z)
- **isdecimal**(): Comprueba si todos los caracteres de una cadena son decimales (0-9)
- **sdigit**(): Comprueba si todos los caracteres de una cadena son números (0-9 y algunos otros caracteres unicode para los números)
- **isnumeric**(): Comprueba si todos los caracteres de una cadena están relacionados con los números o el número (al igual que isdigit(), solo acepta más símbolos, como ⅟2)
- **isidentifier**(): comprueba si hay un identificador válido: comprueba si una cadena es un nombre de variable válido
- **islower**(): Comprueba si todos los caracteres del alfabeto de la cadena son minúsculas
- **isupper**(): Comprueba si todos los caracteres del alfabeto de la cadena son mayúsculas
- **strip**(): Elimina todos los caracteres dados a partir del principio y del final de la cadena
- **replace**(): Reemplaza la subcadena por una cadena determinada
- **split**(): Divide la cadena, usando una cadena o espacio dado como separador

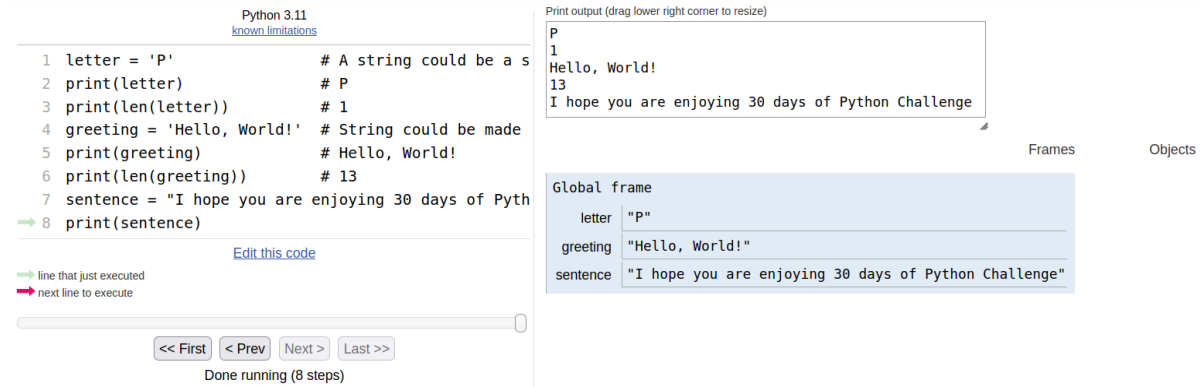
- **swapcase**(): Convierte todos los caracteres mayúsculas en minúsculas y todos los caracteres minúsculos en caracteres mayúsculas
- **startwith**(): Comprueba si la cadena comienza con la cadena especificada

Para comprobar la longitud de una cadena utilice el método **len()**.

---

## Crear una cadena

```
letter = 'P'              # A string could be a single character or a bunch of texts
print(letter)             # P
print(len(letter))        # 1
greeting = 'Hello, World!' # String could be made using a single or double quote,"Hello, World!"
print(greeting)           # Hello, World!
print(len(greeting))      # 13
sentence = "I hope you are enjoying 30 days of Python Challenge"
print(sentence)
```




## Lo que PythonTutor muestra y lo que omite.

### 1. `letter` — Variable, Namespace y Heap

`letter` no almacena el string `'P'` directamente. `letter` es una entrada en el namespace del Global Frame, que es un diccionario donde la **clave** es el nombre `"letter"` (string) y el valor es una dirección de memoria que apunta a un objeto `PyUnicodeObject` en el heap.

Ese objeto en el heap **contiene el dato `'P'` directamente** — no hay un segundo puntero. La cadena corta vive en el campo `data` del mismo objeto.

`globals()` es simplemente la función builtin que **retorna ese dict**, no es el namespace en sí — el namespace existe siempre, `globals()` solo te da acceso a él desde Python.

## ¿Cómo se usa el método `globals()`?

`globals()` es una función incorporada que te devuelve **el diccionario real** que Python está usando como namespace global en ese preciso momento.

Aquí tienes cómo se usa y para qué sirve:

**A. Para "espiar" tus variables**

Si corres esto en tu script:

Python
```
nombre = "Python"
edad = 30

# Pedimos el diccionario del namespace
espia = globals()

print(espia['nombre']) # Imprime: Python
print(espia['edad'])   # Imprime: 30
```

**B. Para ver qué hay "bajo el capó"**

Si imprimes `globals()` al principio de un archivo vacío, verás que no está vacío. Python guarda ahí cosas automáticas como:

- `__name__`: Indica si el script se está ejecutando directamente.
- `__doc__`: El docstring del archivo.
- `__builtins__`: Donde viven funciones como `print` o `len`.

```
globals() dict (namespace del Global Frame)
┌──────────────────────────────────┐
│  clave: "letter"                 │
│  valor: puntero → 0xAAA          │
│                                  │
│  clave: "greeting"               │
│  valor: puntero → 0xBBB          │
└──────────────────────────────────┘
        │                  │
        ▼                  ▼
    Heap 0xAAA         Heap 0xBBB
   ┌──────────┐       ┌──────────┐
   │ob_refcnt=1│      │ob_refcnt=1│
   │ob_type=str│      │ob_type=str│
   │length = 1 │      │length = 13│
   │data = 'P' │      │data = ... │
   └──────────┘       └──────────┘
```

El **namespace** en este contexto es simplemente el diccionario concreto que Python usa en ese momento para resolver nombres de variables.

Cuando escribís `letter = 'P'`, Python hace dos cosas separadas:

**En el heap** crea un objeto `PyUnicodeObject` que contiene el dato `'P'`. Ese objeto vive en una dirección de memoria, digamos `0xAAA`.

**En el namespace del Global Frame** agrega una entrada al diccionario: la clave es el string `"letter"` y el valor es el puntero `0xAAA`. Nada más. El namespace no sabe qué es `'P'`, solo sabe que `"letter"` apunta a algo en `0xAAA`.

Entonces el namespace en este caso específico es ese diccionario interno del Global Frame. Cuando escribís `globals()` en Python, te devuelve exactamente ese dict. Podés comprobarlo:

python

```python
letter = 'P'

print(globals()['letter'])  # 'P'

print(id(letter) == id(globals()['letter']))  # True
```

Ambas expresiones resuelven al mismo objeto en el heap porque `globals()['letter']` y `letter` son exactamente la misma entrada del mismo diccionario.

La palabra "namespace" puede sonar abstracta pero en CPython no lo es: es literalmente un `dict` de Python. El Global Frame tiene uno, cada llamada a función crea otro (el frame local con su propio dict), y los módulos tienen el suyo. Cuando Python evalúa un nombre como `greeting`, lo que hace internamente es buscar la clave `"greeting"` en ese diccionario, obtener el puntero que tiene como valor, y usar ese puntero para acceder al objeto real en el heap.

Lo que Python Tutor muestra como la columna "Frames" con sus filas de nombre→valor es precisamente eso: está renderizando el contenido de ese diccionario, aunque lo presenta visualmente como una tabla en lugar de mostrarte la sintaxis `{}` de un dict.

**2. Evaluación de `print(len(greeting))` — orden y búsqueda**

Tomando `print(len(greeting))` como ejemplo, Python lo evalúa de adentro hacia afuera:

```
print( len( greeting ) )
  3      2      1
```

**Paso 1 — resuelve `greeting`:** Busca en el namespace del Global Frame (el dict de `globals()`). Encuentra la clave `"greeting"`, obtiene el puntero `0xBBB` y lo pasa como argumento a `len`.

**Paso 2 — resuelve y ejecuta `len`:** Busca `"len"` siguiendo el orden LEGB:

Local → no está

Global → no está

Builtin → ✓ encontrado en __builtins__

`len` es una función C en el módulo `builtins`. Recibe el puntero al objeto `0xBBB`, lee el campo `length` del `PyUnicodeObject` (ya calculado al crear el objeto, no itera) y retorna el entero `13`.

**Paso 3 — resuelve y ejecuta `print`:** Igual que `len`, se busca en `builtins`. Recibe el entero `13` y lo escribe en stdout.

Orden de búsqueda LEGB (para print y len):

```
Orden de búsqueda LEGB (para print y len):
┌──────────────────────────────────────┐
│ 1. Local    → frame actual     → NO  │
│ 2. Enclosing → func contenedora → NO  │
│ 3. Global   → globals() dict   → NO  │
│ 4. Builtin → __builtins__      → ✓ SÍ │
└──────────────────────────────────────┘
```

### 3. `greeting` — Layout de memoria, indexación y eliminación

`greeting` apunta a la dirección base del objeto en el heap (`0xBBB`). Dentro de ese objeto hay un puntero `data` que apunta al buffer contiguo de caracteres. Para `'Hello, World!'` (todo ASCII) CPython usa **1 byte por carácter**.

Para acceder a `greeting[i]` el cálculo en C es:

dirección del char = data_ptr + i × 1 byte

**No existe un objeto Python "contador de índice"** — esa aritmética ocurre en la capa C del intérprete al evaluar `[]`. El índice es una variable local en C, no vive en el heap.

```
Heap objeto PyUnicodeObject en 0xBBB
┌──────────────────────────────────────┐
│ ob_refcnt = 1                        │
│ ob_type   → &PyUnicode_Type          │
│ length    = 13                       │
│ data_ptr  → 0xCCC ──────────────┐    │
└─────────────────────────────────┼────┘
                                  │
                                  ▼
Buffer contiguo en 0xCCC (1 byte por char):
┌───┬───┬───┬
```

**¿Cuándo se elimina?**

Cada objeto tiene `ob_refcnt`. Cuando hacés `greeting = 'otro'`:

- El objeto en `0xBBB` decrementa su contador.
- Si llega a **0**, el allocator lo libera **inmediatamente** — CPython no espera al garbage collector.

Los **strings cortos e internados** (`'P'`, identificadores, literales de código) tienen su `ob_refcnt` artificialmente alto o viven en un pool interno — **nunca llegan a 0 durante la ejecución**, por eso no se eliminan.

**En el Global Frame** tenés:

"greeting" → 0xBBB

`0xBBB` es la dirección base del objeto `PyUnicodeObject` en el heap. Ese objeto tiene varios campos internos, y uno de ellos es `data_ptr` que vale `0xCCC`.

**Para llegar al carácter en posición `[1]` el recorrido es:**

greeting → ve al namespace, obtiene 0xBBB
0xBBB → ve al heap, es el PyUnicodeObject
PyUnicodeObject → lee el campo data_ptr, obtiene 0xCCC
0xCCC + 1×1 → 0xCCD
*(0xCCD) → 'e'

Son **dos saltos de puntero**, no uno. El Global Frame te da `0xBBB`. Con `0xBBB` llegás al objeto. Dentro del objeto leés `data_ptr` para obtener `0xCCC`. Recién ahí hacés la aritmética `0xCCC + índice × 1 byte` para llegar al carácter concreto.

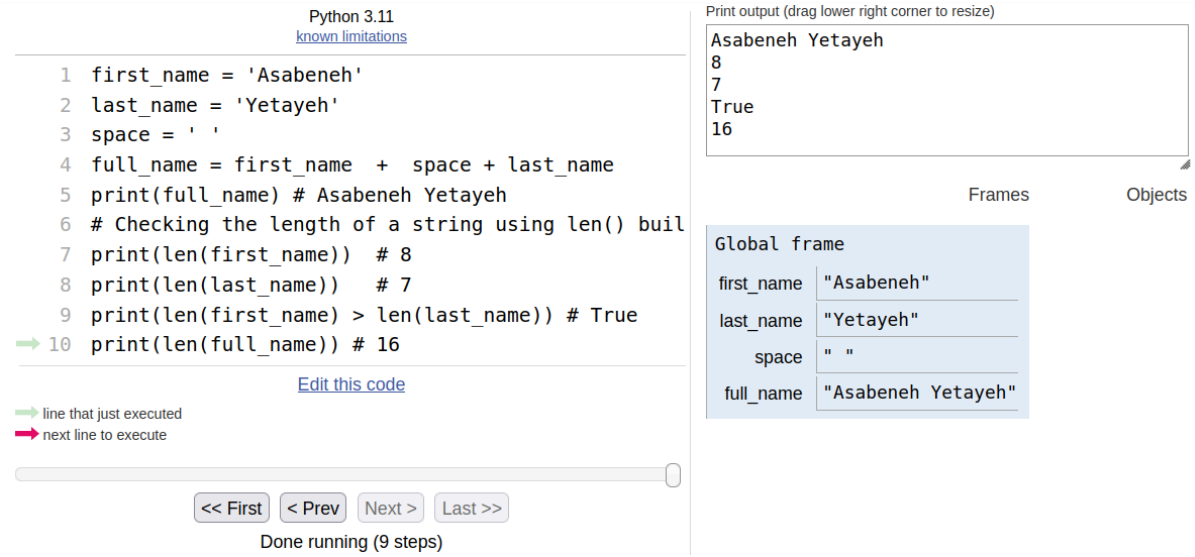
La imagen lo muestra exactamente así: `greeting[4]` necesitó primero llegar a `0xBBB`, leer `data_ptr = 0xCCC`, y luego calcular `0xCCC + 4×1 = 0xCD0`, que contiene `'o'`.

El Global Frame nunca apunta directo al buffer. Siempre pasás por el objeto intermedio.

## Concatenación de cuerdas

Podemos conectar las cuerdas. La fusión o conexión de cuerdas se llama concatenación. Vea el ejemplo a continuación:

```
first_name = 'Asabeneh'
last_name = 'Yetayeh'
space = ' '
full_name = first_name  +  space + last_name
print(full_name) # Asabeneh Yetayeh
# Checking the length of a string using len() built-in function
print(len(first_name))  # 8
print(len(last_name))   # 7
print(len(first_name) > len(last_name)) # True
print(len(full_name)) # 16
```




## Setup — Global Frame antes de la concatenación

globals() dict

```
┌──────────────────────────────────┐
| "first_name" → puntero → 0xAAA  |
| "last_name"  → puntero → 0xBBB  |
| "space"      → puntero → 0xCCC  |
└──────────────────────────────────┘


Heap:
0xAAA → PyUnicodeObject { data: "Asabeneh", length: 8 }
0xBBB → PyUnicodeObject { data: "Yetayeh",  length: 7 }
0xCCC → PyUnicodeObject { data: " ",        length: 1 }
```

## Evaluación de `full_name = first_name + space + last_name`

El operador `+` entre strings **no se busca con LEGB** — correcto. Es un operador del lenguaje. El intérprete lo compila como el bytecode `BINARY_OP`, que internamente llama al método dunder `__add__` del objeto string.

La expresión se evalúa **de izquierda a derecha** en dos pasos porque `+` es asociativo izquierdo:

first_name + space + last_name
= (first_name + space) + last_name
= resultado_intermedio + last_name
= full_name

### Paso 1 — `first_name + space`

1. Resuelve "first_name" en globals() → obtiene puntero 0xAAA
2. Resuelve "space" en globals() → obtiene puntero 0xCCC
3. Bytecode BINARY_OP llama:
   (0xAAA).__add__(0xCCC)
   → PyUnicode_Concat(0xAAA, 0xCCC)
   → crea NUEVO objeto en heap 0xDDD
   → copia "Asabeneh" + " " en buffer contiguo nuevo

Heap nuevo 0xDDD → { data: "Asabeneh ", length: 9 }

```
0xAAA ["Asabeneh"] ──┐
                     ├──→ PyUnicode_Concat → 0xDDD ["Asabeneh "]
0xCCC [" "]        ──┘
```

El objeto `0xDDD` es **temporal** — no está asignado a ninguna variable todavía.

### Paso 2 — `resultado_intermedio + last_name`

1. Toma el puntero del resultado anterior 0xDDD (en el stack de evaluación)

2. Resuelve "last_name" en globals() → obtiene puntero 0xBBB

3. Bytecode BINARY_OP llama:

   (0xDDD).__add__(0xBBB)

→ PyUnicode_Concat(0xDDD, 0xBBB)

→ crea NUEVO objeto en heap 0xEEE

→ copia "Asabeneh " + "Yetayeh" en buffer contiguo nuevo

Heap nuevo 0xEEE → { data: "Asabeneh Yetayeh", length: 16 }

```
0xDDD ["Asabeneh "] ──┐
                      ├──→ PyUnicode_Concat → 0xEEE ["Asabeneh Yetayeh"]
0xBBB ["Yetayeh"]  ───┘
```

**Paso 3 — Asignación `full_name = …`**

```
globals() dict agrega:
┌──────────────────────────────────┐
│ "full_name" → puntero → 0xEEE    │
└──────────────────────────────────┘
```

El objeto temporal `0xDDD` (`"Asabeneh "`) ahora tiene `ob_refcnt = 0` → **se libera inmediatamente**.

### Estado final del heap

```
globals() dict
┌──────────────────────────────────┐
│ "first_name" → 0xAAA             │
│ "last_name"  → 0xBBB             │
│ "space"      → 0xCCC             │
│ "full_name"  → 0xEEE             │
└──────────────────────────────────┘

Heap:
0xAAA → ["Asabeneh"]          ob_refcnt=1  (vive)
0xBBB → ["Yetayeh"]           ob_refcnt=1  (vive)
0xCCC → [" "]                 ob_refcnt=1  (vive, posiblemente internado)
0xDDD → ["Asabeneh "]         ob_refcnt=0  (LIBERADO)
0xEEE → ["Asabeneh Yetayeh"] ob_refcnt=1  (vive)
```

### Resumen del mecanismo de `+`

| Concepto | Detalle |
|---|---|
| `+` no usa LEGB | Es un operador compilado a bytecode `BINARY_OP` |
| Método invocado | `__add__` del objeto string izquierdo |
| Función C real | `PyUnicode_Concat()` |
| Resultado | Siempre un objeto **nuevo** en el heap |
| Asociatividad | Izquierda a derecha — genera un objeto intermedio temporal |
| Objeto temporal | Se libera inmediatamente al terminar la expresión |

---

_

**Cadena Multilínea:**

En la mayoría de los lenguajes de programación, si intentas cerrar una cadena de texto (`string`) después de presionar "Enter", el código se rompe y lanza un error.

Las cadenas multilínea (o *triple-quoted strings*) existen para:

- **Mantener el formato:** Conservan los saltos de línea y los espacios exactamente como los escribes.
- **Evitar caracteres de escape:** No necesitas usar `\n` para crear una nueva línea o preocuparte por si usas comillas simples o dobles dentro del texto.
- **Documentación:** Son la base de los *docstrings* (documentación interna de funciones y clases).

| Tipo | Ejemplo | Uso común |
|---|---|---|
| **Triple simple** | `''' Texto '''` | Textos largos o cuando el texto contiene comillas dobles. |
| **Triple doble** | `""" Texto """` | Estándar oficial de Python para documentación (*docstrings*). |

**Explicación de tu ejemplo**

Analicemos tu código paso a paso:

multiline_string = '''I am a teacher and enjoy teaching.

I didn't find anything as rewarding as empowering people.

That is why I created 30 days of python.'''

print(multiline_string)

1. **multiline_string =**: Estás creando una variable para almacenar el texto.
2. **Las comillas triples ('''')**: Le avisan a Python: *"Oye, todo lo que venga a continuación es parte del mismo texto, incluso si salto de línea, hasta que veas otras tres comillas iguales"*.
3. **El contenido**:
    - Fíjate que usaste I didn't. En una cadena normal con comillas simples ('I didn't'), esto causaría un error porque Python pensaría que la cadena termina en la n. Con comillas triples, esto no es un problema.
4. **print(multiline_string)**: Al imprimirlo, Python respetará los saltos de línea que hiciste al escribir el código.

**Un detalle importante**

Aunque las usamos para crear variables de texto largo, si pones una cadena de comillas triples suelta en tu código (sin asignarla a una variable), Python la ignora. Por eso se usan mucho para escribir **comentarios de varias líneas**, aunque técnicamente Python las procesa como una cadena de texto que no se guarda en ningún lado.

> **Tip de pro:** Si alguna vez necesitas escribir un párrafo que incluya tanto comillas simples (') como dobles ("), las comillas triples son tu mejor aliado para no volverte loco escapando caracteres con barras invertidas (\).

---

## Secuencias de escape en cuerdas

En Python y otros lenguajes de programación \ seguido de un carácter es una secuencia de escape. Veamos los personajes de escape más comunes:

- \n: nueva línea
- \t: Tab media(8 espacios)
- \\: Barra de espalda
- \': Cita única (')
- \": Doble cita (")

**Resumen de Sintaxis**

| Secuencia | Nombre Real | ¿Para qué sirve? |
|---|---|---|
| `\n` | New Line | Baja a la siguiente línea. |
| `\t` | Horizontal Tab | Alinea texto en columnas. |
| `\\` | Backslash | Muestra una barra invertida `\` real. |
| `\"` | Double Quote | Muestra `"` sin romper la cadena de texto. |
| `\'` | Single Quote | Muestra `'` sin romper la cadena de texto. |

## 1. El Salto de Línea (`\n`)

- **Teoría:** La `n` viene de *newline*. Es el equivalente a presionar la tecla "Enter" en tu teclado mientras escribes un documento.
- **Sintaxis:** Se coloca justo donde quieres que el texto se rompa y continúe abajo.
- **Análisis del ejemplo:**
- Python

```
print('I hope everyone is enjoying the Python Challenge.\nAre you ?')
```

- 
- Aquí, el texto antes de `\n` aparece en la primera fila, y "Are you ?" aparece en la segunda. Sin el `\n`, todo saldría en una sola línea kilométrica.

## 2. La Tabulación (`\t`)

- **Teoría:** La `t` viene de *tab*. No es simplemente "un espacio", sino que mueve el texto hacia la siguiente "parada de tabulación" (generalmente cada 4 u 8 espacios). Es ideal para alinear columnas sin tener que calcular espacios a mano.
- **Sintaxis:** `\t` entre las palabras que quieres separar.
- **Análisis del ejemplo:**
- Python

```
print('Days\tTopics\tExercises')

print('Day 1\t5\t5')
```

- 
- Como ves en tu código, el uso de `\t` hace que "Days", "Topics" y "Exercises" queden perfectamente alineados verticalmente con los números de abajo, creando una estructura de tabla limpia.

## 3. La Barra Invertida (`\\`)

- **Teoría:** Aquí es donde la cosa se pone meta. Como la barra invertida `\` es el carácter de escape, si tú quieres que el usuario **vea** una barra invertida en la pantalla, Python se confunde pensando que vas a dar una instrucción. Para imprimir una, tienes que "escapar al escapador".
- **Sintaxis:** Escribes dos veces la barra para obtener una.
- **Análisis del ejemplo:**
- Python

print('This is a backslash symbol (\\)')

- 
- **Resultado:** `This is a backslash symbol (\)`. La primera barra le dice a Python "prepárate", y la segunda le dice "el carácter que quiero mostrar es una barra".

## 4. Comillas Simples y Dobles (`\'` y `\"`)

- **Teoría:** En Python, las cuerdas (strings) se encierran en comillas. Si abres una cuerda con comillas dobles (`"`) y quieres poner otra comilla doble adentro, Python pensará que la frase terminó ahí y el resto del código dará error.
- **Sintaxis:** `\"` para mostrar una comilla doble, `\'` para una simple.
- **Análisis del ejemplo:**
- Python

print("In every programming language it starts with \"Hello, World!\"")

- Aquí usamos `\"` para que las comillas que rodean a *Hello, World!* se impriman literalmente y no cierren el comando `print` antes de tiempo.

Python 3.11
known limitations

```
1 print('I hope everyone is enjoying the Python Chall
2 print('Days\tTopics\tExercises') # adding tab space
3 print('Day 1\t5\t5')
4 print('Day 2\t6\t20')
5 print('Day 3\t5\t23')
6 print('Day 4\t1\t35')
7 print('This is a backslash  symbol (\\)') # To writ
8 print('In every programming language it starts with
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (8 steps)

Print output (drag lower right corner to resize)

```
I hope everyone is enjoying the Python Challenge.
Are you ?
Days    Topics  Exercises
Day 1   5       5
Day 2   6       20
Day 3   5       23
Day 4   1       35
This is a backslash  symbol (\)
In every programming language it starts with "Hello, World!"
```

Frames Objects

---

_

## 1. Formato estilo antiguo — operador %

El operador % toma un string con marcadores de posición y una tupla de valores, y los combina en un nuevo string. Es el mecanismo más antiguo, heredado de C.

**Sintaxis:**

python
'string con %marcador' % (valor1, valor2)

**Marcadores disponibles:**

| Marcador | Tipo |
|---|---|
| `%s` | String (o cualquier objeto, llama `str()`) |
| `%d` | Entero |
| `%f` | Float |
| `%.Nf` | Float con N decimales fijos |

python
```
# --- Ejemplo 1: solo strings ---
first_name = 'Asabeneh'        # string en heap, first_name apunta a él
```

```python
last_name = 'Yetayeh'          # ídem
language = 'Python'           # ídem

formated_string = 'I am %s %s. I teach %s' % (first_name, last_name, language)
# El operador % recibe el string plantilla y la tupla (first_name, last_name, language)
# Sustituye cada %s por el str() del objeto correspondiente, en orden
# Crea un nuevo objeto string en el heap con el resultado
print(formated_string)
# → I am Asabeneh Yetayeh. I teach Python

# --- Ejemplo 2: strings y números ---
radius = 10                   # objeto int en heap
pi = 3.14                     # objeto float en heap
area = pi * radius ** 2       # evalúa primero radius**2=100, luego 3.14*100=314.0
                              # crea nuevo objeto float 314.0 en heap

formated_string = 'The area of circle with a radius %d is %.2f.' % (radius, area)
# %d convierte radius a entero formateado → "10"
# %.2f formatea area con exactamente 2 decimales → "314.00"
# Crea nuevo objeto string con el resultado
print(formated_string)
# → The area of circle with a radius 10 is 314.00.

# --- Ejemplo 3: lista como argumento ---
python_libraries = ['Django', 'Flask', 'NumPy', 'Matplotlib', 'Pandas']
# python_libraries es una variable que apunta a un objeto list en el heap

formated_string = 'The following are python libraries:%s' % (python_libraries)
# %s llama str() sobre el objeto list completo → lo convierte a su representación textual
print(formated_string)
# → The following are python libraries:['Django', 'Flask', 'NumPy', 'Matplotlib', 'Pandas']
```

## 2. Formato nuevo estilo — `str.format()`

Introducido en Python 3. Las llaves `{}` actúan como marcadores de posición. `format()` es un método del objeto string que retorna un nuevo string con los valores insertados.

**Sintaxis:**

python
```python
'string con {} marcadores'.format(valor1, valor2)
'con formato {:.2f}'.format(valor_float)   # 2 decimales
```
python
```python
first_name = 'Asabeneh'
last_name = 'Yetayeh'
language = 'Python'
```

```
formated_string = 'I am {} {}. I teach {}'.format(first_name, last_name, language)
# format() recorre las {} en orden y sustituye cada una con str() del argumento
# Retorna nuevo objeto string en el heap
print(formated_string)
# → I am Asabeneh Yetayeh. I teach Python

a = 4
b = 3

print('{} + {} = {}'.format(a, b, a + b))   # a+b=7  se evalúa antes de pasarse a format()
print('{} - {} = {}'.format(a, b, a - b))   # a-b=1
print('{} * {} = {}'.format(a, b, a * b))   # a*b=12
print('{} / {} = {:.2f}'.format(a, b, a / b)) # a/b=1.333..., {:.2f} lo redondea a 1.33
print('{} % {} = {}'.format(a, b, a % b))   # a%b=1  (resto de división)
print('{} // {} = {}'.format(a, b, a // b)) # a//b=1 (división entera)
print('{} ** {} = {}'.format(a, b, a ** b)) # a**b=64

# → 4 + 3 = 7
# → 4 - 3 = 1
# → 4 * 3 = 12
# → 4 / 3 = 1.33
# → 4 % 3 = 1
# → 4 // 3 = 1
# → 4 ** 3 = 64

radius = 10
pi = 3.14
area = pi * radius ** 2                 # 314.0

formated_string = 'The area of a circle with a radius {} is {:.2f}.'.format(radius, area)
# {} inserta radius tal cual → "10"
# {:.2f} formatea area con 2 decimales → "314.00"
print(formated_string)
# → The area of a circle with a radius 10 is 314.00.
```

## 3. f-Strings — Interpolación directa (Python 3.6+)

El mecanismo más moderno. La `f` antes de las comillas indica que el string es evaluado en tiempo de ejecución: las expresiones dentro de `{}` se resuelven directamente en el contexto actual, sin pasar argumentos explícitos.

**Sintaxis:**

python
```
f'texto {variable} texto {expresion} texto'
f'float con decimales {valor:.2f}'
```

A diferencia de `%` y `.format()`, las `{}` de un f-string pueden contener **cualquier expresión Python válida**, no solo variables.

python
```
a = 4
b = 3

print(f'{a} + {b} = {a + b}')    # resuelve a=4, b=3, a+b=7 en el momento de crear el string
print(f'{a} - {b} = {a - b}')    # a-b=1
print(f'{a} * {b} = {a * b}')    # a*b=12
print(f'{a} / {b} = {a / b:.2f}') # a/b=1.333..., :.2f lo formatea a 1.33
print(f'{a} % {b} = {a % b}')    # a%b=1
print(f'{a} // {b} = {a // b}')  # a//b=1
print(f'{a} ** {b} = {a ** b}')  # a**b=64
```

### 4. Strings como secuencias — Desempaquetado

Un string en Python es una secuencia ordenada de caracteres. Esto permite asignar cada carácter a una variable distinta en una sola línea, del mismo modo que se desempaqueta una lista o tupla.

**Sintaxis:**

python
```
v1, v2, v3, ... = string   # cantidad de variables debe coincidir exactamente con len(string)
```

Si la cantidad de variables no coincide con la longitud del string, Python lanza `ValueError`.

python
```
language = 'Python'        # string de longitud 6

a, b, c, d, e, f = language # desempaqueta: asigna cada char en orden a cada variable
                            # equivale a: a='P', b='y', c='t', d='h', e='o', f='n'
                            # cada variable queda apuntando al objeto char en el heap

print(a)  # P
print(b)  # y
print(c)  # t
print(d)  # h
print(e)  # o
print(f)  # n
```

---

–

## Acceso a Strings por Índice y Slicing en CPython

---

## 1. Acceso por índice positivo

En programación el conteo comienza en cero. El primer carácter está en índice `0`, el último en índice `len(string) - 1`. Internamente Python calcula la dirección del carácter como `data_ptr + i × bytes_por_char`.

**Sintaxis:**

python

```
string[indice]       # índice desde 0 hasta len-1
```

python

```
language = 'Python'          # variable en globals() apunta a objeto string en heap

first_letter = language[0]    # data_ptr + 0 → 'P'

print(first_letter)          # P

second_letter = language[1]   # data_ptr + 1 → 'y'

print(second_letter)         # y

last_index = len(language) - 1  # len() lee el campo length del objeto → 6, resta 1 → 5

last_letter = language[last_index]  # data_ptr + 5 → 'n'

print(last_letter)           # n
```

Cuando hacés `language[0]`, Python **no copia** el carácter a nueva memoria en el caso general. Lo que hace es:

1. Va al `PyUnicodeObject` en el heap
2. Lee el puntero al buffer de caracteres
3. Calcula `data_ptr + 0 × 1` (si es Latin-1)
4. Lee el byte en esa posición → tiene el código de `'P'`

Pero acá viene lo importante: **ese byte no es un objeto Python**. Es un número crudo en memoria (ej: 80 = código ASCII de 'P').

Para poder usarlo como `first_letter = language[0]`, Python necesita **devolverte un objeto Python**, porque en Python todo es un objeto. Entonces sí crea (o reutiliza) un `PyUnicodeObject` nuevo en el heap que representa el string `'P'`.

**Pero con un truco crucial: internamiento (interning).**

Para strings de un solo carácter (y muchos strings cortos comunes), CPython los tiene **pre-creados y cacheados** en una tabla interna al arrancar. Entonces cuando indexás y el resultado es `'P'`, Python no crea nada nuevo, simplemente **devuelve una referencia al objeto `'P'` que ya existía en esa tabla**.

```
language[0]
    │
    ▼
data_ptr + 0  →  byte 80 (valor crudo, no es objeto)
    │
    ▼
CPython busca en tabla interna de chars
    │
    ▼
first_letter ──→ PyUnicodeObject 'P' @ 0x7f... (pre-existente, no nuevo)
```

Entonces la respuesta directa: **se crea/reutiliza un objeto nuevo**, no se apunta directo al buffer, porque el buffer contiene bytes crudos, no objetos Python. Pero gracias al interning de caracteres individuales, en la práctica casi nunca se crea memoria nueva, se reutiliza la misma dirección siempre.

Podés verificarlo:

python
```
language = 'Python'
print(id(language[0]) == id(language[0]))  # True — misma dirección, mismo objeto
```

## 2. Índice negativo

Python permite indexar desde el final usando números negativos. `-1` es el último carácter. Internamente se resuelve como `length + indice_negativo`.

**Sintaxis:**

python

```
string[-1]    # último carácter

string[-N]    # N-ésimo carácter desde el final
```

python

```
language = 'Python'


last_letter = language[-1]    # -1 → 6 + (-1) = índice 5 → data_ptr + 5 → 'n'
```

```
print(last_letter)          # n

second_last = language[-2]   # -2 → 6 + (-2) = índice 4 → data_ptr + 4 → 'o'

print(second_last)          # o
```

## 3. Slicing — Subcadenas

Permite extraer una porción del string. **Siempre crea un nuevo objeto string en el heap** — no es una vista del original.

**Sintaxis:**

python

```
string[start:stop]       # desde start hasta stop-1 (stop no incluido)

string[start:]           # desde start hasta el final

string[:stop]            # desde el inicio hasta stop-1

string[start:stop:step]  # con salto entre caracteres
```

**Modelo de memoria para `language[0:3]`**

```
globals() dict
┌──────────────────────────────────┐
| "language"    → puntero 0xAAA   |
| "first_three" → puntero 0xBBB   |  ← nueva entrada tras el slice
└──────────────────────────────────┘

Heap objeto original 0xAAA:
data_ptr → 0xF000

Buffer en 0xF000 (1 byte por char, ASCII):
┌────┬────┬────┬────┬────┬────┐
| P  | y  | t  | h  | o  | n  |
└────┴────┴────┴────┴────┴────┘
0xF000 0xF001 0xF002 0xF003 0xF004 0xF005
  i=0    i=1    i=2    i=3    i=4    i=5

Slice [0:3] copia chars en índices 0,1,2:
→ nuevo objeto 0xBBB en heap
→ nuevo buffer 0xC000 con copia de 'P','y','t'

┌────┬────┬────┐
| P  | y  | t  |
└────┴────┴────┘
0xC000 0xC001 0xC002
```

Cada carácter siguiente se encuentra sumando el tamaño del puntero por carácter al anterior:

P → 0xF000
y → 0xF000 + 1 byte = 0xF001
t → 0xF001 + 1 byte = 0xF002
h → 0xF002 + 1 byte = 0xF003
o → 0xF003 + 1 byte = 0xF004
n → 0xF004 + 1 byte = 0xF005

> El tamaño del paso en bytes depende del string completo: 1 byte si todos los chars son ASCII, 2 bytes si hay chars Unicode hasta U+FFFF, 4 bytes si hay chars fuera de ese rango. El slice hereda el mismo ancho.

python
language = 'Python'

```
first_three = language[0:3]   # copia chars en índices 0,1,2 → nuevo objeto "Pyt"
print(first_three)            # Pyt

last_three = language[3:6]    # copia chars en índices 3,4,5 → nuevo objeto "hon"
print(last_three)             # hon

last_three = language[-3:]    # -3 → índice 3, hasta el final → "hon"
print(last_three)             # hon

last_three = language[3:]     # desde índice 3 hasta el final → "hon"
print(last_three)             # hon
```

Con slice el comportamiento es **distinto** al índice simple, porque el resultado es un string de múltiples caracteres, y esos no están internados.

**Lo que hace Python:**

1. Calcula el rango: índices 0, 1, 2
2. Va al buffer de `language` y lee los bytes en esas posiciones: `80, 121, 116` (P, y, t)
3. **Crea un `PyUnicodeObject` nuevo en el heap**
4. Crea un **buffer nuevo** y **copia** esos 3 bytes ahí
5. Devuelve referencia al objeto nuevo

```
language @ heap:
┌──────────────────────────────────┐
│ PyUnicodeObject 'Python'         │
│   length: 6                      │
│   data_ptr ──────────────────┐   │
└──────────────────────────────┼───┘
                               ▼
                    buffer: [P][y][t][h][o][n]
                              ↑  ↑  ↑
                           lee estos 3 bytes
                                  │
                                  │ copia
                                  ▼
resultado @ heap (NUEVO objeto):
┌──────────────────────────────────┐
│ PyUnicodeObject 'Pyt'            │
│   length: 3                      │
│   data_ptr ──────────────────┐   │
└──────────────────────────────┼───┘
                               ▼
                    buffer nuevo: [P][y][t]
```

Podés verificar que es un objeto completamente nuevo:

```python
language = 'Python'
sliced = language[0:3]
print(id(language) == id(sliced))  # False — direcciones distintas
```

**¿Por qué no apunta directo al buffer original?**

Porque los strings en Python son **inmutables e independientes**. Si `'Pyt'` apuntara al buffer de `'Python'` y después `'Python'` fuera liberado de memoria (refcount = 0), `'Pyt'` quedaría apuntando a memoria inválida. La copia garantiza que cada objeto string es autosuficiente y maneja su propio ciclo de vida.

Esto es diferente a lenguajes como Go o Rust donde un slice puede ser una **vista** del buffer original sin copiar. En CPython, cada string es siempre dueño de su propio buffer.

---

## 4. Revertir un string

Caso especial de slice con `step = -1`: recorre el buffer de derecha a izquierda, copiando cada carácter en orden inverso en un nuevo objeto string.

**Sintaxis:**

python
string[::-1]  # start y stop omitidos = recorre todo; step=-1 = de atrás hacia adelante
python
greeting = 'Hello, World!'

print(greeting[::-1])  # step=-1 → recorre el buffer desde índice 12 hasta 0
# crea nuevo objeto string con los chars en orden inverso
# → !dlroW ,olleH

---

## 5. Saltar caracteres con `step`

El tercer parámetro del slice es el paso: cuántos índices avanzar entre cada carácter seleccionado. Internamente equivale a `data_ptr + i × step × bytes_por_char`.

**Sintaxis:**

python
string[start:stop:step]  # toma un carácter cada `step` posiciones
python
language = 'Python'

pto = language[0:6:2]  # índices 0, 2, 4 → chars 'P', 't', 'o'
# índice 0: 0xF000 → 'P'
# índice 2: 0xF000 + 2×1 = 0xF002 → 't'
# índice 4: 0xF000 + 4×1 = 0xF004 → 'o'
# crea nuevo objeto string "Pto" en heap
print(pto)  # Pto

---

## Métodos de cuerda

Hay muchos métodos de cadena que nos permiten formatear cadenas. Vea algunos de los métodos de cadena en el siguiente ejemplo:

- capitalize(): Convierte el primer carácter de la cadena en mayúscula

challenge = 'thirty days of python'
print(challenge.capitalize()) # 'Thirty days of python'

- count(): devuelve las apariciones de subcadena en string, count(substring, start=.., end=..). El inicio es una indexación inicial para contar y el final es el último índice a contar.

```
challenge = 'thirty days of python'
print(challenge.count('y')) # 3
print(challenge.count('y', 7, 14)) # 1,
print(challenge.count('th')) # 2`
```

- endswith(): Comprueba si una cadena termina con un final especificado

```
challenge = 'thirty days of python'
print(challenge.endswith('on'))   # True
print(challenge.endswith('tion')) # False
```

- expandtabs(): Reemplaza el carácter de la pestaña con los espacios, el tamaño predeterminado de la pestaña es 8. Se necesita argumento de tamaño de pestaña

```
challenge = 'thirty\tdays\tof\tpython'
print(challenge.expandtabs())   # 'thirty  days    of      python'
print(challenge.expandtabs(10)) # 'thirty    days      of        python'
```

- find(): Devuelve el índice de la primera aparición de una subcadena, si no se encuentran retornos -1

```
challenge = 'thirty days of python'
print(challenge.find('y'))  # 5
print(challenge.find('th')) # 0
```

- rfind(): Devuelve el índice de la última ocurrencia de una subcadena, si no se encuentran retornos -1

```
challenge = 'thirty days of python'
print(challenge.rfind('y'))  # 16
print(challenge.rfind('th')) # 17
```

- format(): cadena de formatos en una salida más bonita
  Más información sobre el formato de cadena comprobar este [enlace]

```
first_name = 'Asabeneh'
last_name = 'Yetayeh'
age = 250
job = 'teacher'
country = 'Finland'
sentence = 'I am {} {}. I am a {}. I am {} years old. I live in {}.'.format(first_name, last_name, job, age, country)
print(sentence) # I am Asabeneh Yetayeh. I am 250 years old. I am a teacher. I live in Finland.

radius = 10
pi = 3.14
area = pi * radius ** 2
result = 'The area of a circle with radius {} is {}'.format(str(radius), str(area))
```

```
print(result) # The area of a circle with radius 10 is 314
```

- index(): Devuelve el índice más bajo de una subcadena, los argumentos adicionales indican el índice de inicio y finalización (predeterminado 0 y longitud de cadena - 1). Si no se encuentra la subcadena, aumenta un valorError.

```
challenge = 'thirty days of python'
sub_string = 'da'
print(challenge.index(sub_string))  # 7
print(challenge.index(sub_string, 9)) # error
```

- ridex(): Devuelve el índice más alto de una subcadena, los argumentos adicionales indican el índice de inicio y finalización (predeterminado 0 y longitud de cadena - 1)

```
challenge = 'thirty days of python'
sub_string = 'da'
print(challenge.rindex(sub_string))  # 7
print(challenge.rindex(sub_string, 9)) # error
print(challenge.rindex('on', 8)) # 19
```

- isalnum(): Comprueba el carácter alfanumérico

```
challenge = 'ThirtyDaysPython'
print(challenge.isalnum()) # True

challenge = '30DaysPython'
print(challenge.isalnum()) # True

challenge = 'thirty days of python'
print(challenge.isalnum()) # False, space is not an alphanumeric character

challenge = 'thirty days of python 2019'
print(challenge.isalnum()) # False
```

- isalpha(): Comprueba si todos los elementos de cadena son caracteres alfabéticos (a-z y A-Z)

```
challenge = 'thirty days of python'
print(challenge.isalpha()) # False, space is once again excluded
challenge = 'ThirtyDaysPython'
print(challenge.isalpha()) # True
num = '123'
print(num.isalpha())      # False
```

- isdecimal(): Comprueba si todos los caracteres de una cadena son decimales (0-9)

```
challenge = 'thirty days of python'
print(challenge.isdecimal())  # False
challenge = '123'
```

```
print(challenge.isdecimal())  # True
challenge = '\u00B2'
print(challenge.isdigit())   # True
challenge = '12 3'
print(challenge.isdecimal())  # False, space not allowed
```

- isdigit(): Comprueba si todos los caracteres de una cadena son números (0-9 y algunos otros caracteres unicode para los números)

```
challenge = 'Thirty'
print(challenge.isdigit()) # False
challenge = '30'
print(challenge.isdigit())   # True
challenge = '\u00B2'
print(challenge.isdigit())   # True
```

- isnumeric(): Comprueba si todos los caracteres de una cadena están relacionados con los números o el número (al igual que isdigit(), solo acepta más símbolos, como ⅟2)

```
num = '10'
print(num.isnumeric()) # True
num = '\u00BD' # ½
print(num.isnumeric()) # True
num = '10.5'
print(num.isnumeric()) # False
```

- isidentifier(): comprueba si hay un identificador válido: comprueba si una cadena es un nombre de variable válido

```
challenge = '30DaysOfPython'
print(challenge.isidentifier()) # False, because it starts with a number
challenge = 'thirty_days_of_python'
print(challenge.isidentifier()) # True
```

- islower(): Comprueba si todos los caracteres del alfabeto de la cadena son minúsculas

```
challenge = 'thirty days of python'
print(challenge.islower()) # True
challenge = 'Thirty days of python'
print(challenge.islower()) # False
```

- isupper(): Comprueba si todos los caracteres del alfabeto de la cadena son mayúsculas

```
challenge = 'thirty days of python'
print(challenge.isupper()) #  False
challenge = 'THIRTY DAYS OF PYTHON'
```

```
print(challenge.isupper()) # True
```

- Join(): Devuelve una cadena concatenada

```
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
result = ' '.join(web_tech)
print(result) # 'HTML CSS JavaScript React'
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
result = '# '.join(web_tech)
print(result) # 'HTML# CSS# JavaScript# React'
```

- strip(): Elimina todos los caracteres dados a partir del principio y del final de la cadena

```
challenge = 'thirty days of pythoonnn'
print(challenge.strip('noth')) # 'irty days of py'
```

- replace(): Reemplaza la subcadena por una cadena determinada

```
challenge = 'thirty days of python'
print(challenge.replace('python', 'coding')) # 'thirty days of coding'
```

- split(): Divide la cadena, usando una cadena o espacio dado como separador

```
challenge = 'thirty days of python'
print(challenge.split()) # ['thirty', 'days', 'of', 'python']
challenge = 'thirty, days, of, python'
print(challenge.split(', ')) # ['thirty', 'days', 'of', 'python']
```

- title(): Devuelve una cadena de código

```
challenge = 'thirty days of python'
print(challenge.title()) # Thirty Days Of Python
```

- swapcase(): Convierte todos los caracteres mayúsculas en minúsculas y todos los caracteres minúsculos en caracteres mayúsculas

```
challenge = 'thirty days of python'
print(challenge.swapcase())   # THIRTY DAYS OF PYTHON
challenge = 'Thirty Days Of Python'
print(challenge.swapcase())  # tHIRTY dAYS oF pYTHON
```

- startwith(): Comprueba si la cadena comienza con la cadena especificada

```
challenge = 'thirty days of python'
print(challenge.startswith('thirty')) # True

challenge = '30 days of python'
print(challenge.startswith('thirty')) # False
```